

Shiv Nadar University Chennai

Name of Student	KR Srinidhi Rajamane	Reg. No. 24011101059
Degree & Branch	B.Tech Artificial Intelligence & Data Science Section A	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026– 27

Experiment 2

Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

1. Objective

The objective of this experiment is to implement a Multi-Layer Perceptron using TensorFlow/Keras for multi-class image classification on the Fashion-MNIST dataset. Students will learn image preprocessing (flattening, normalization, one-hot encoding), MLP model construction, training and evaluation, and automated hyperparameter optimization using Randomized Search with cross-validation.

2. Learning Outcomes

Students will be able to:

1. Explain the architecture of an MLP.
2. Preprocess image datasets.
3. Build and train an MLP.
4. Evaluate classification performance.
5. Perform automated hyperparameter optimization.
6. Recommend the best model configuration.

3. Background Theory

An MLP consists of an input layer, one or more hidden layers, and an output layer. Each layer applies an affine transformation followed by a non-linear activation:

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = f(z^{(l)})$$

where $W^{(l)}$ and $b^{(l)}$ are the weight matrix and bias vector of layer l , and f is a non-linear activation function such as ReLU.

For multi-class classification, the output layer uses the Softmax activation to produce a probability distribution over the classes:

$$\hat{y} = \text{Softmax}(z)$$

The model is trained by minimizing the categorical cross-entropy loss:

$$L = - \sum_i y_i \log(\hat{y}_i)$$

where y_i is the one-hot true label and $\hat{y}_i$ is the predicted probability for class i .

Recommended architecture:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

Unlike the Single Layer Perceptron studied in Experiment 1, which can only represent a single linear decision boundary, the hidden layers of an MLP allow the network to learn non-linear decision boundaries, making it capable of solving problems (such as multi-class image classification) that are not linearly separable.

4. Dataset Description

Dataset: Fashion-MNIST

Training Images	60,000
Testing Images	10,000
Classes	10
Image Size	28×28 (grayscale)

Class Labels

T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, Ankle boot.

5. Image Preprocessing

Fashion-MNIST images are stored as 28×28 matrices. Dense layers require a one-dimensional feature vector, so each image is flattened:

$$28 \times 28 = 784$$

$$\begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} \rightarrow [10, 20, 30, 40]$$

Preprocessing steps performed:

1. Load Fashion-MNIST.
2. Display sample images.
3. Flatten each 28×28 image into a 784-length vector.
4. Normalize pixel values from $[0, 255]$ to $[0, 1]$.
5. Convert integer labels into one-hot vectors.

Verified tensor shapes:

Tensor	Before	After
Training images	(60000, 28, 28)	(60000, 784)
Testing images	(10000, 28, 28)	(10000, 784)
Training labels	(60000,)	(60000, 10) one-hot
Testing labels	(10000,)	(10000, 10) one-hot

Pixel range before normalization: $[0, 255]$. Pixel range after normalization: $[0.00, 1.00]$.

6. Experimental Procedure

Task 1: Dataset Exploration

- Loaded Fashion-MNIST via `keras.datasets.fashion_mnist`.
- Printed dataset dimensions for all four splits.
- Displayed ten sample images with class labels (Fig. 1).
- Plotted the class distribution across all 10 categories (Fig. 2).

Task 2: Data Preprocessing

- Flattened all images from 28×28 to 784-dimensional vectors.
- Normalized pixel values to $[0, 1]$.
- Verified and printed tensor shapes before and after preprocessing (Section 5).

Task 3: Model Construction

Constructed a baseline MLP:

- Input layer: 784 units.
- Dense(128, ReLU) – hidden layer 1.
- Dense(64, ReLU) – hidden layer 2.

- Dense(10, Softmax) – output layer.

Model summary (109,386 trainable parameters):

Layer	Output Shape	Parameters
hidden_layer_1 (Dense)	(None, 128)	100,480
hidden_layer_2 (Dense)	(None, 64)	8,256
output_layer (Dense)	(None, 10)	650
Total		109,386

Task 4: Model Training

Compiled with Adam optimizer, categorical cross-entropy loss, and accuracy metric. Trained for 20 epochs with batch size 32 (validation split 10%). Training accuracy rose from 82.21% (epoch 1) to 93.5% by epoch 19, while validation accuracy plateaued around 87–88%, with a training time of approximately 86.9 seconds.

Task 5: Model Evaluation

Computed Accuracy, Precision, Recall, F1-score, Confusion Matrix and Classification Report on the 10,000-sample test set (Section 8).

7. Hyperparameter Optimization

Automated hyperparameter optimization was performed using a manually implemented **Randomized Search with 5-fold cross-validation**, evaluated over a 10,000-sample stratified subset of the training data (chosen to keep search time tractable in Colab; the final model was retrained on the full 60,000-sample training set).

Search Space

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation Function	ReLU, Tanh, Sigmoid
Dropout Rate	0.0, 0.2, 0.5

Procedure

1. Built a baseline MLP model (Section 6, Task 3).
2. Defined the hyperparameter search space above.

3. Randomly sampled 20 configurations and evaluated each with 5-fold cross-validation on the 10,000-sample search subset.
4. Recorded the best-performing hyperparameter combination (Section 9).
5. Retrained the model with the optimized hyperparameters on the full 60,000-sample training set.
6. Evaluated the optimized model on the 10,000-sample test set.
7. Compared optimized vs. baseline performance (Section 9).

8. Baseline Model – Test Set Results

Metric	Value
Accuracy	0.8760 (87.60%)
Precision (macro)	0.8780 (87.80%)
Recall (macro)	0.8760 (87.60%)
F1-score (macro)	0.8740 (87.40%)

Per-class performance was strongest on Trouser (F1 = 0.97), Sandal (F1 = 0.97) and Bag (F1 = 0.96), and weakest on Shirt (F1 = 0.66), which was frequently confused with T-shirt/top and Pullover.

9. Results

Best Hyperparameters (from Randomized Search, 5-fold CV)

Hidden Layers	3
Hidden Neurons	256
Learning Rate	0.001
Batch Size	16
Optimizer	Adam
Activation Function	Sigmoid
Epochs	20
Dropout	0.2
Cross-validation Accuracy	0.8561 (85.61%)
Testing Accuracy	0.8846 (88.46%)

Performance Comparison

Metric	Baseline	Optimized
Accuracy	0.8760	0.8846
Precision	0.8780	0.8867
Recall	0.8760	0.8846
F1-score	0.8740	0.8819
Training Time (s)	86.93	345.55

The optimized model improved the test accuracy by 0.86 percentage points and the F1-score by 0.79 percentage points over the baseline.

10. Discussion

1. Which optimization method was used?

A Randomized Search over the defined hyperparameter space, evaluated with 5-fold cross-validation on a 10,000-sample stratified subset of the training data.

2. Which hyperparameters were selected?

3 hidden layers, 256 neurons per layer, learning rate 0.001, batch size 16, Adam optimizer, Sigmoid activation, 20 epochs, dropout rate 0.2.

3. Did optimization improve performance?

Yes, modestly. The test accuracy improved from 87.60% to 88.46%, the precision from 87.80% to 88.67%, recall from 87.60% to 88.46%, and F1-score from 87.40% to 88.19%. The gain is consistent but small in absolute terms.

4. Which hyperparameter had the greatest impact?

The depth and width of the network (hidden layers = 3, neurons = 256) appear to matter the most.

5. Compare Grid Search and Randomized Search.

Grid Search exhaustively evaluates every combination in the search space, guaranteeing the best combination *within the grid* is found, but its cost grows multiplicatively with the number of hyperparameters and candidate values and so it is computationally infeasible to fully cross-validate. Randomized Search instead samples a fixed budget of configurations (20, in this experiment) from the space, trading the guarantee of finding the single best combination for the ability to explore a much larger, higher-dimensional space within a practical time budget. In high-dimensional search spaces like this one, Randomized Search is generally preferred because most hyperparameters have limited individual effect on performance, so random sampling tends to discover near-optimal regions of the space with far fewer trials than an exhaustive grid would need.

6. Recommend the best MLP configuration.

Based on the search results, the recommended configuration is: 2–3 hidden layers, 128–256 neurons per layer, ReLU or Sigmoid activation, learning rate 0.001, Adam optimizer, batch size 16–32, dropout 0.2, trained for 20 epochs. This balances the accuracy gains of a deeper network with the increased training time.

11. Source Code

The complete implementation was carried out in Python using TensorFlow/Keras, NumPy, Pandas, Matplotlib, Seaborn and Scikit-learn. The full, executable source code (organized cell-wise, exactly as it was run) is available on GitHub:

https://github.com/Srinidhi-Krishna/Deep-Learning-Lab/blob/main/Lab_2/DL_LAB_2.ipynb

12. Generated Plots with Interpretation

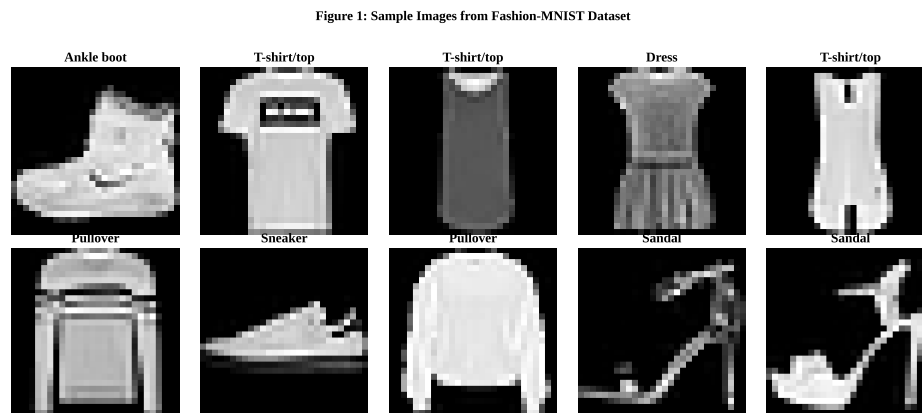


Figure 1: Ten sample images from the Fashion-MNIST training set with class labels.

Interpretation: The ten samples show clear inter-class shape variation (e.g., trousers vs. bags vs. sandals), confirming the dataset's suitability for a classification task requiring spatial feature discrimination.

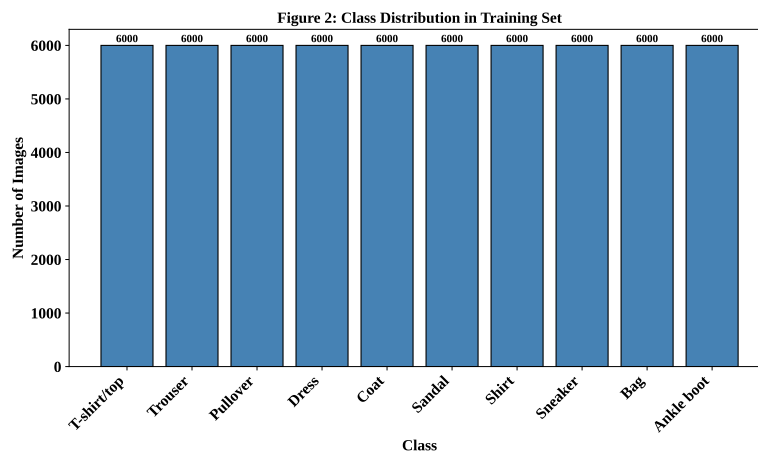


Figure 2: Class distribution across the 10 Fashion-MNIST categories in the training set.

Interpretation: All 10 classes contain exactly 6,000 training images, confirming a perfectly balanced dataset. This eliminates the need for class re-weighting or resampling strategies during training.

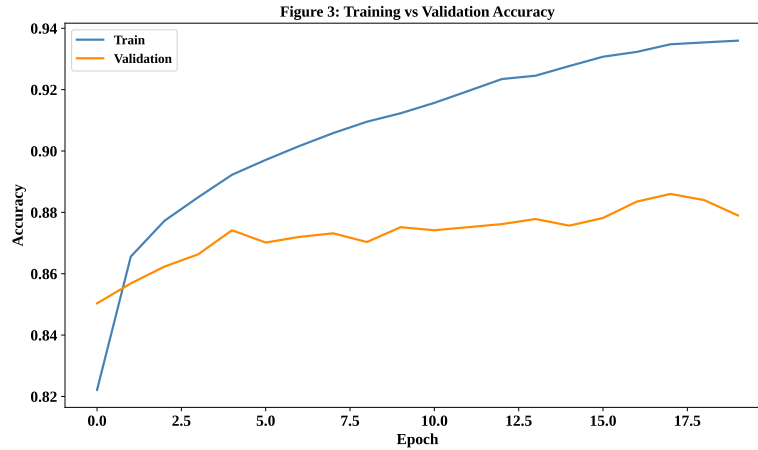


Figure 3: Training vs. validation accuracy across 20 epochs (baseline model).

Interpretation: Training accuracy rises steadily while validation accuracy plateaus, indicating the model is learning discriminative features but with a widening train-validation gap suggestive of mild overfitting in later epochs.

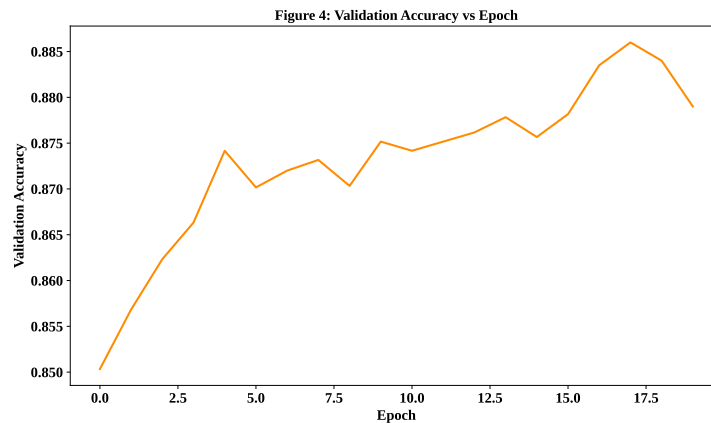


Figure 4: Validation accuracy vs. epoch (baseline model).

Interpretation: Validation accuracy climbs quickly at first and then flattens into a noisy plateau, suggesting that the baseline architecture is close to its representational ceiling on this dataset without further regularization or capacity increases.

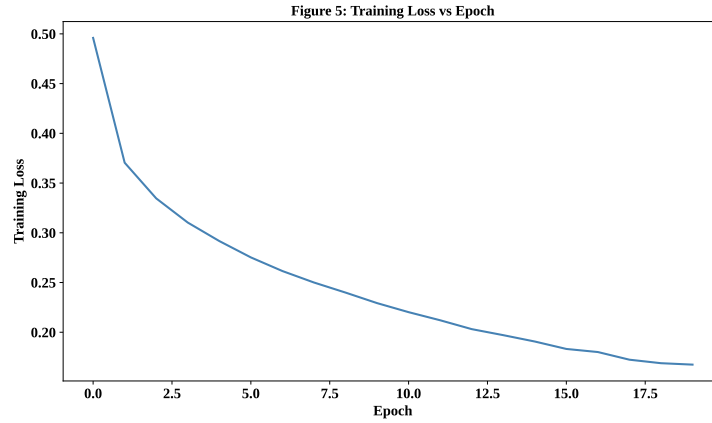


Figure 5: Training loss vs. epoch (baseline model).

Interpretation: Categorical cross-entropy loss decreases monotonically, confirming stable convergence of the Adam optimizer with no oscillation or divergence across the 20 training epochs.

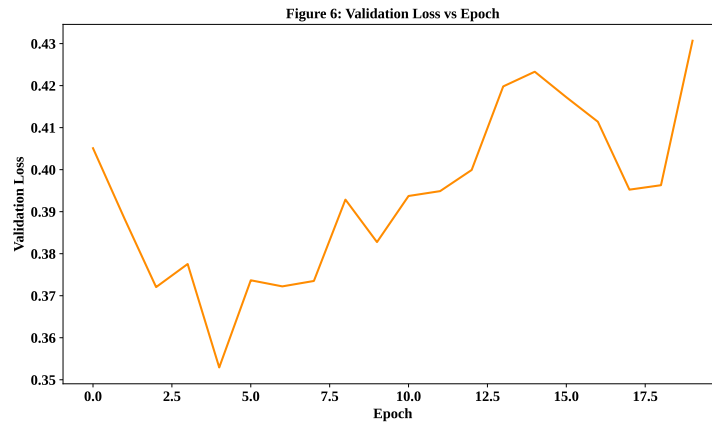


Figure 6: Validation loss vs. epoch (baseline model).

Interpretation: Validation loss decreases alongside training loss at first but then begins to rise again even as training loss keeps falling which is a classic early sign of overfitting.

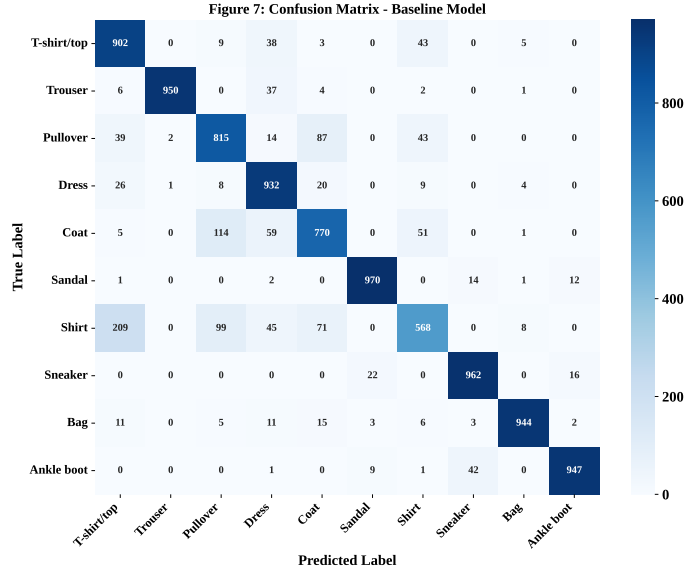


Figure 7: Confusion matrix on the test set (baseline model, 10,000 samples).

Interpretation: Most misclassifications occur between visually similar classes particularly Shirt vs. T-shirt/top and Shirt vs. Pullover but it is consistent with their overlapping silhouette shapes in grayscale, and explains why Shirt has the lowest per-class F1-score (0.66) of all ten categories.

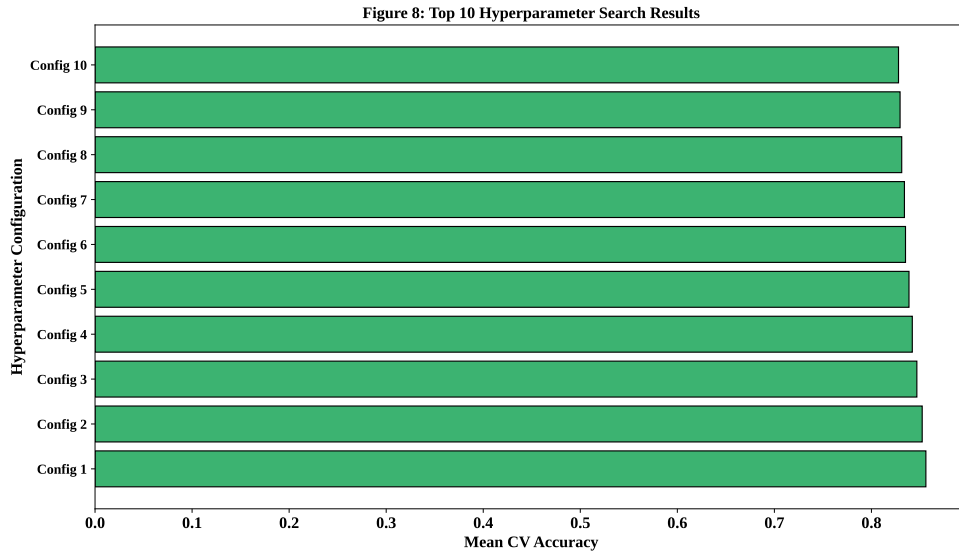


Figure 8: Top 10 hyperparameter configurations ranked by mean 5-fold cross-validation accuracy.

Interpretation: The top 10 configurations cluster tightly indicating that several distinct hyperparameter combinations (differing in depth, activation, and optimizer) achieve near-optimal performance, suggesting the search space contains multiple good local optima rather than one single dominant configuration.

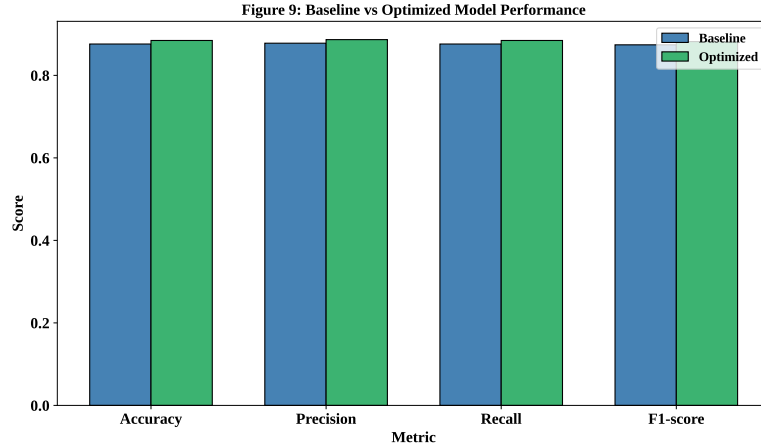


Figure 9: Baseline vs. optimized model performance across four evaluation metrics.

Interpretation: The optimized model outperforms the baseline consistently but modestly across all four metrics (accuracy, precision, recall, F1-score), confirming that automated hyperparameter tuning yields a measurable gain over the manually chosen defaults.

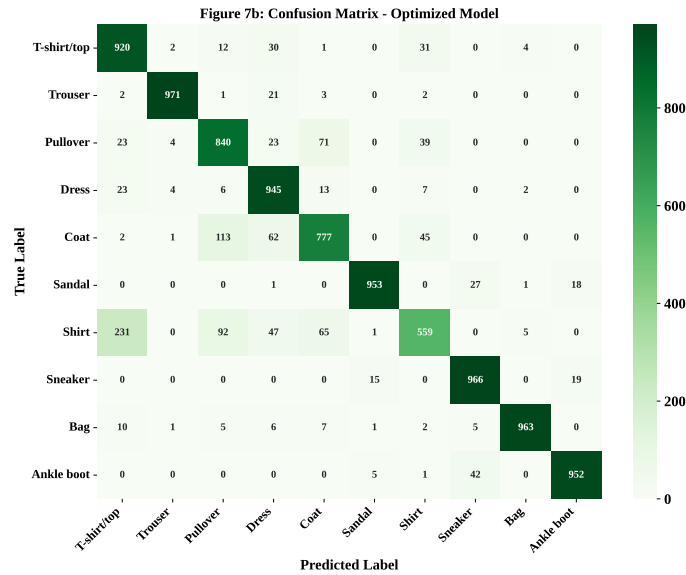


Figure 10: Confusion matrix on the test set (optimized model, 10,000 samples).

Interpretation: The optimized model shows a reduction in off-diagonal confusion in the Pullover/Coat/Dress cluster, indicating better learned feature separability after tuning.

13. Conclusion

A Multi-Layer Perceptron was implemented in TensorFlow/Keras and evaluated on the Fashion-MNIST dataset. The baseline architecture ($784 \rightarrow 128 \rightarrow 64 \rightarrow 10$, ReLU/Softmax) achieved 87.60% test accuracy. Automated hyperparameter optimization via a manually implemented Randomized Search with 5-fold

cross-validation identified a deeper, wider configuration (3 hidden layers, 256 neurons, Sigmoid activation, dropout 0.2) that improved test accuracy to 88.46%, at roughly $4\times$ the training cost. Both models struggled most on the Shirt class, which is visually similar to T-shirt/top, Pullover and Coat, indicating that further gains would likely require convolutional (rather than purely dense) architectures capable of exploiting local spatial structure.

14. References

1. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
2. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
3. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
4. H. Xiao, K. Rasul and R. Vollgraf, “Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms,” 2017.
5. TensorFlow/Keras Documentation.
6. Scikit-learn Documentation: RandomizedSearchCV, KFold.